2025 IEEE 64th Conference on Decision and Control (CDC)
December 10-12, 2025. Rio de Janeiro, Brazil

Action-Priority Driven Policy Optimization for Flexible Job Shop Scheduling Problem

Wenjun Zheng, Weilin Cai, Wei Qu, Jianfeng Mao*, *Member, IEEE*

2025 IEEE 64th Conference on Decision and Control (CDC) | 979-8-3315-2627-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/CDC57313.2025.11312736

***Abstract*— Efficient scheduling significantly impacts productivity and operational performance, particularly in flexible resource allocation scenarios. The Flexible Job Shop Scheduling Problem (FJSP) extends traditional scheduling by allowing each operation to be processed on multiple machines, enhancing practical applicability but increasing computational complexity. Existing methods, ranging from exact solvers to heuristic/metaheuristic algorithms, either incur high computational costs or yield suboptimal solutions. Recent reinforcement learning (RL)-based approaches attempt a balance between optimality and efficiency; however, intrinsic characteristics of FJSP remain underutilized for better solution quality. This paper introduces the Action-Priority Driven Policy Optimization (APDPO) method, explicitly designed to address three critical challenges: achieving superior solutions, improving interpretability via an intuitive policy representation, and enhancing computational efficiency through direct, gradient-free policy updates. A policy improvement theorem ensures performance enhancement under specified conditions. Comparative evaluations confirm that APDPO outperforms existing approaches by delivering better solution quality, reduced computational requirements, and greater decision transparency.**

I. Introduction

Efficient scheduling is vital in modern industries, as it directly impacts productivity, cost, and operational performance. The Flexible Job Shop Scheduling Problem (FJSP) is widely applied in areas such as automotive manufacturing, semiconductor fabrication, and logistics [1], [2]. In FJSP, each operation can be processed on multiple alternative machines, extending the classical Job Shop Scheduling Problem (JSP) to multi-purpose machines [3], [4] or multiprocessors [5]. Since JSP is NP-hard [6], FJSP is even more challenging. With the rise of automation and customization, FJSP is now increasingly regarded as a distinct problem rather than a mere generalization of JSP [7].

Approaches to the FJSP span exact, heuristic, metaheuristic, and learning-based methods. Exact methods, such as branch-and-bound [8] and solvers like Gurobi [9] and Google OR-Tools [10], guarantee optimality but are computationally prohibitive. Heuristics, including the shifting bottleneck [11] and dispatch rules, improve efficiency at the cost of solution quality. Metaheuristics, such as tabu search [12], genetic algorithms [13], and their recent variants [14], [15], [16], enhance performance yet remain unsuitable for real-time use. Deep reinforcement learning (DRL) [17], [18] offers a promising balance between quality and efficiency with online adaptability. Early DRL methods relied on dispatch rules [19], [20], while newer approaches such as MPGN [21] and HGNN [15] provide finer control but still exhibit significant optimality gaps. DANIEL [16] further narrows this gap, though solutions still lag behind Google OR-Tools [10]. Despite progress, existing methods remain challenged by suboptimal instance-level solutions, limited interpretability, and high computational cost. These shortcomings motivate the following key questions: **Q1)** Can individual problem solutions be further improved? **Q2)** Does a simple yet sufficiently expressive policy representation exist? **Q3)** Can the policy be updated more directly and effectively?

In this paper, we address these key questions by introducing the Action-Priority Driven Policy Optimization (APDPO) method. To answer **Q2)**, we design a policy representation inspired by the sequential and one-time decision-making nature of state-action pairs in FJSP. The policy is formulated as a linear vector, where each entry denotes the priority of a corresponding action. Regarding policy updates, we propose a direct adjustment mechanism that reinforces high-performing actions while proportionally reducing the probability of others. Unlike neural network-based policy approximations, this update process is gradient-free, effectively addressing **Q3)**. For **Q1)**, we establish a policy improvement theorem based on the proposed policy representation and update mechanism, ensuring policy enhancement under specific conditions. The experimental results demonstrate that APDPO significantly outperforms existing RL approaches and achieves performance much closer to that of OR-Tools.

The rest of the paper is organized as follows: Section II introduces the problem description of FJSP and the reinforcement learning model. Section III presents the proposed APDPO method in detail. Numerical results are provided in Section IV. Finally, Section V concludes the paper; proofs are summarized in Section VI. Notations are aggregated in

Wenjun Zheng and Wei Qu are with the School of Data Science, The Chinese University of Hong Kong, Shenzhen (CUHK-Shenzhen), Guangdong 518172, China. (email: {wenjunzheng, weiqu}@link.cuhk.edu.cn)

Weilin Cai is with the School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen (CUHK-Shenzhen), Guangdong 518172, China. (email: weilincai@link.cuhk.edu.cn)

Jianfeng Mao is with the School of Data Science, The Chinese University of Hong Kong, Shenzhen (CUHK-Shenzhen), Guangdong 518172, China. He is also with the Shenzhen Research Institute of Big Data, Guangdong 518172, China. (email: jfmao@cuhk.edu.cn)

This work was supported in part by the National Natural Science Foundation of China (NSFC) under grants 72394362 and U1733102; in part by the Guangdong Provincial Key Laboratory of Big Data Computing, The Chinese University of Hong Kong, Shenzhen (CUHK-Shenzhen) under grant 2024SC0003; in part by the Shenzhen Science and Technology Innovation Committee under the Shenzhen Stability Science Program grant 2024SC0010; in part by the Shenzhen Research Institute of Big Data under grant J00120250001; in part by CUHK-Shenzhen under grant PF.01.000404; and in part by the Shenzhen Science and Technology Innovation Bureau under the Low-Altitude Airspace Strategic Program Portfolio.

* Corresponding author: Jianfeng Mao.

Authorized licensed use limited to: Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on April 08,2026 at 12:30:36 UTC from IEEE Xplore. Restrictions apply.

Section I of our supplement material[1].

II. Problem Formulation

A. Flexible Job Shop Scheduling Problem

In the FJSP, there are a set of jobs $\mathcal{J} = \{J_1, \cdots, J_{\mathbb{n}}\}$ and machines $\mathcal{M} = \{M_1, \cdots, M_{\mathbb{m}}\}$, where jobs need to be scheduled on machines and $\mathbb{n}, \mathbb{m}$ denotes the number of jobs and machines respectively. Each job J_j consists of an ordered sequence of operations $(O_{j,1}, O_{j,2}, \ldots, O_{j,\mathbb{k}_j})$, where $\mathbb{k}_j$ denotes the total number of operations in job J_j. Unlike the conventional JSP, the FJSP introduces flexibility by allowing each operation $O_{j,i}$ to be processed on any machine from its eligible subset $\mathcal{M}_{j,i} \subseteq \mathcal{M}$. To ensure a feasible schedule, several constraints must be met: **1)** Each machine can process at most one operation at a time. **2)** Each operation must be assigned to exactly one machine from its eligible set $\mathcal{M}_{j,i}$. **3)** Operations within a job must follow their predefined sequence, meaning that $O_{j,i+1}$ cannot start until its preceding operation $O_{j,i}$ has finished execution.

The makespan $C_{\max}$ represents the total completion time of the schedule and is defined as the latest finishing time among all operations. The objective of this paper is to minimize the makespan.

B. Reinforcement Learning Model

The FJSP considered in this work is formulated as a finite-horizon Markov Decision Process (MDP) comprising stages, states, actions, dynamics, and rewards. The value function given policy π is denoted as $V^{\pi}(s)$. The components of the MDP are structured as follows.

1) Stage and state: A stage $t \in \{0, 1, \cdots, T-1\}$ corresponds to a decision point when available operation machine assignments exist, T is the horizon. In the FJSP context, these stages occur at discrete time points when machine-operation assignments become feasible. The system state at stage t is formally defined as $s_t \doteq (\mathbf{J}_t, \mathbf{M}_t, \mathbf{W}_t, \mathbb{T}_t)$, comprising four critical elements, reified in Fig. 1 in the supplement material.

The Job progress matrix $\mathbf{J}_t \in \mathbb{N}^{\mathbb{n} \times 2}$ tracks the status of $\mathbb{n}$ jobs through two attributes per row. For any job, the first column records the index of its earliest unprocessed operation (0 if the job is completed) while the second column acts as a binary flag, with 0 indicating an idle job and 1 denoting in-progress processing. Additionally, the machine availability vector $\mathbf{M}_t \in \{0, 1\}^m$ reflects the status of m machines: 0 signifies an idle machine, and 1 represents occupancy. Processing information are captured by the work set $\mathbf{W}_t$ which contains 4-tuples $(m, j, i, p_{m,j,i})$ describing machine M_m's current engagement with operation $O_{j,i}$ of job J_j, where $p_{m,j,i} \in \mathbb{R}^+$ quantifies the remaining processing time. The scalar $\mathbb{T}_t \in \mathbb{R}^+$ serves as a continuous-time system clock, initialized to 0 at $t = 0$, to synchronize temporal progression across scheduling decisions. Notably, at the final stage $t = T$, $\mathbb{T}_T$ equals the makespan.

[1] https://whiskychoy.github.io/CDC2025APD/SUPP.pdf

2) Action: An action $a \doteq (m, j, i)$ represents the assignment of a machine M_m to an operation $O_{j,i}$. The feasible action space denoted as $\mathcal{A}(s_t)$ defines the set of machine operation pairs that can be scheduled at the current stage t given state s_t. An action is eligible for selection only if its target operation's preceding operations are completed, with its designated machine idle.

3) Dynamics: Given an action a_t and the current state s_t, the next state is obtained by updating the job progress matrix $\mathbf{J}_t$, the machine availability vector $\mathbf{M}_t$, and the work set $\mathbf{W}_t$ accordingly, while keeping the system clock unchanged. If no actions are available after this modification, the system clock $\mathbb{T}_t$ advances to the earliest time at which an operation is completed and at least one action becomes available. The elements $\mathbf{J}_t, \mathbf{M}_t$, and $\mathbf{W}_t$ in the state are then will be modified again. One may refer to Subsections II-A&B in the supplement material for the detailed update rule.

4) Reward: The reward decomposes the makespan based on the system clock difference between consecutive stages, reflecting the elapsed time. Given the current state s_t and action a_t, the next state s_{t+1} is uniquely determined by the system dynamics. The reward is defined as $r(s_t, a_t) = \mathbb{T}_t - \mathbb{T}_{t+1}$. Such reward is always non-positive. Over a trajectory $\tau = (s_0, a_0, r_0, ..., a_{T-1}, r_{T-1}, s_T)$, the accumulative reward is computed as:

$$\sum_{t=0}^{T-1} r(s_t, a_t) = \sum_{t=0}^{T-1} (\mathbb{T}_t - \mathbb{T}_{t+1}) = -\mathbb{T}_T = -C_{\max}.$$

Thus, the reward structure inherently drives the system toward maximizing the return $-C_{\max}$, which corresponds to minimizing the makespan.

III. Algorithm

This section presents the Action-Priority Driven Policy Optimization method for solving the FJSP. APDPO leverages the Actor-Critic framework [22], where both the policy and value function are crucial for solving the problem. While neural networks are commonly used for their representational power in both components, APDPO adopts a different approach by directly representing the policy as a vector, with its length corresponding to the total number of possible actions.

The remaining parts are structured as follows: Subsection III-A introduces the policy representation. Subsection III-B presents the policy update rule and provides relevant theorems on policy improvement. Subsection III-C offers an in-depth interpretation of the proposed APDPO method.

A. Policy Representation

We propose a linear policy representation. Let $|\mathcal{A}|$ represent the total number of possible actions, where $|\mathcal{A}| \leq \mathbb{m} \times \sum_{j=1}^{\mathbf{n}} \mathbb{k}_j$. We define the *action-priority vector* Φ, which has a length of $|\mathcal{A}|$:

$$\Phi := \left[\phi_1, \cdots, \phi_a, \cdots, \phi_{|\mathcal{A}|}\right].$$

Here, each element is nonnegative, and ϕ_a represents the priority value of action a. Let $\mathbb{I}(s, a)$ be an indicator function,

Authorized licensed use limited to: Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on April 08,2026 at 12:30:36 UTC from IEEE Xplore. Restrictions apply.

where

$$\mathbb{I}(s,a) = \begin{cases} 1, & \text{if action } a \text{ is valid for state } s, \\ 0, & \text{otherwise.} \end{cases}$$

Then, the stochastic policy π at state s is given by

$$\pi(a|s) = \frac{\phi_a}{\sum_{a'} \mathbb{I}(s,a')\phi_{a'}}, \quad a \in \mathcal{A}(s). \tag{1}$$

Since all elements of Φ are nonnegative, it follows that $\pi(a|s) \geq 0$ and that the probability distribution satisfies $\sum_{a\in\mathcal{A}(s)} \pi(a|s) = 1$. Therefore, the policy defined in (1) is well-posed. During training, a stochastic policy π is employed, whereas after training, a deterministic policy μ, obtained from the *action-priority vector* Φ by selecting the action with the highest probability, is used:

$$\mu(s) \doteq \arg\max_{a\in\mathcal{A}(s)} \pi(a|s) = \arg\max_{a\in\mathcal{A}(s)} \phi_a. \tag{2}$$

For the deterministic policy μ, it can be fully determined by the priority order according to considering Equation (2).The priority order of actions corresponds to the positions of their values when sorted according to the *action-priority vector*.

B. policy update

In this section, we introduce the policy update rule. Given a policy π with the corresponding *action-priority vector* Φ and a trajectory $\tau := (s_t, a_t, s_{t+1}, ...)$, the updated policy π' and Φ' are given by

$$\phi'_a = \begin{cases} \phi_a + (1-\delta)\sum_{a'\neq a} \mathbb{I}(s_t,a')\phi_{a'}, & \text{if } a = a_t, \\ \delta\phi_a, & \text{otherwise} \end{cases} \tag{3}$$

where δ is the update step size and $\delta \in (0,1)$.

Since all components of Φ are nonnegative, the update rule (3) ensures that Φ' remains nonnegative as well. Under the proposed update rule (3), the performance improves if the step size δ satisfies certain conditions. This result is formally established in Theorem 1. Although these conditions are complex and not directly verifiable, they serve a crucial role in the proof. To address this, we later introduce Theorem 2, which provides more practical, verifiable conditions.

Theorem 1 (Policy Improvement–Version I): Given a policy π, corresponding *action-priority vector* Φ and a trajectory $\tau_t := (s_t, a_t, s_{t+1}, ...)$. Assume $a_t = (m, j, i)$, let $\mathcal{A}_m(s_t) \subseteq \mathcal{A}(s_t)$ denote the set of feasible actions where the machine is fixed as M_m. Similarly, let $\mathcal{A}_{(j,i)}(s_t) \subseteq \mathcal{A}(s_t)$ denote the set of feasible actions where the job and operation are fixed as J_j and $O_{j,i}$, respectively. Suppose $G(\tau_t) > V^\pi(s_t)$, where $G(\tau_t)$ is the return of the trajectory τ_t and $V^\pi(s_t)$ is the state value function under policy π. Then we have new policy π' and its corresponding Φ' where

$$\phi'_a = \begin{cases} \phi_a + (1-\delta)\sum_{a'\neq a} \mathbb{I}(s_t,a')\phi_{a'}, & \text{if } a = a_t \\ \delta\phi_a, & \text{otherwise} \end{cases} \tag{4}$$

and $\delta \in (0,1)$. Then, there exists a constant $\rho \in (0,1)$ such that if $\eta \in (0,\rho)$, there exists $\epsilon \in (0,1)$ such that the following holds: if $\delta \in (\epsilon, 1)$, and δ satisfies condition **CD1**, and $\mathcal{A}_m(s_t), \mathcal{A}_{j,i}(s_t)$ satisfies condition **CD2**, then the updated policy π', along with its corresponding mapping Φ', yields improved performance, i.e., $V^{\pi'}(s_t) > V^\pi(s_t)$.

CD1 $\sum_{t=1}^{t_\tau^+-1} \delta^t \frac{1-\delta\eta}{1-\eta} \mathrm{P}^\pi_{\tau^{(t)}} < \delta(\sum_{t=1}^{t_\tau^+-1} \frac{\eta \mathrm{P}^\pi_{\tau_t|(s_t,a_t)} \mathrm{P}^\pi_{\tau^{(t)}}}{1-(1-\eta)\mathrm{P}^\pi_{\tau_t|(s_t,a_t)}} + \frac{\eta \mathrm{P}^\pi_{\tau|(s_t,a_t)}}{1-(1-\eta)\mathrm{P}^\pi_{\tau_t|(s_t,a_t)}}) + \sum_{t=1}^{t_\tau^+-1} \frac{(1-\mathrm{P}^\pi_{\tau_t|(s_t,a_t)})\mathrm{P}^\pi_{\tau^{(t)}}}{1-(1-\eta)\mathrm{P}^\pi_{\tau_t|(s_t,a_t)}} - \frac{\eta \mathrm{P}^\pi_{\tau|(s_t,a_t)}}{1-(1-\eta)\mathrm{P}^\pi_{\tau_t|(s_t,a_t)}}, \forall \tau \in \mathcal{T}_3, \mathcal{T}_5$

CD2 For any $a \in \mathcal{A}_m(s_t) \cup \mathcal{A}_{(j,i)}(s_t)$, after executing action a, the system clock of the resulting state remains the same as $\mathbb{T}_t$, which is the system clock of s_t.

where $\eta = \frac{\sum_{j\neq a'_0} \mathbb{I}(s'_0,j)\phi_j}{\sum_j \mathbb{I}(s'_0,j)\phi_j}$, $\tau_t \mid (s_t, a_t) = (s_{t+1}, \ldots)$ denotes a truncated trajectory obtained by removing the state-action pair (s_t, a_t), and $\mathrm{P}^\pi_{\tau_t|(s_t,a_t)}$ represents the probability of this truncated trajectory under policy π. The definitions of other potentially unclear terms will be provided in the proof.

The condition **CD1** in Theorem 1 is challenging to verify efficiently. Therefore, we conduct a detailed analysis of this condition and consolidate the findings into Proposition 1.

Proposition 1: If $1 - \frac{\eta}{1-\eta} > \lambda + \kappa$, where

$$\lambda = \frac{\eta \mathrm{P}^\pi_{\tau_t|(s_t,a_t)}}{1-(1-\eta)\mathrm{P}^\pi_{\tau_t|(s_t,a_t)}},$$

$$\kappa = \frac{1}{1-\eta} \frac{\eta}{1-(1-\eta)\mathrm{P}^\pi_{\tau_t|(s_t,a_t)}} \times \left(\frac{1}{\eta} - \frac{\min_{a\in\mathcal{A}(s_t), a\neq a_t} \sum_j \mathbb{I}(s_t,j,a)\phi_j}{\sum_{j\neq a_t} \mathbb{I}(s_t,j)\phi_j} \right),$$

then there exists a constant $\zeta \in (0,1)$, such that when $\delta \in (\zeta, 1)$, **CD1** is satisfied.

Remark 1: By the definition of $\eta = \frac{\sum_{j\neq a_t} \mathbb{I}(s_t,j)\phi_j}{\sum_j \mathbb{I}(s_t,j)\phi_j}$, as well as the trajectory τ_t, states s_t and s_{t+1}, the values of λ and κ can be easily computed. Thus, the condition $1 - \frac{\eta}{1-\eta} > \lambda + \kappa$ can be efficiently verified.

Condition **CD2** can be efficiently verified by determining whether any actions remain available after executing those where either the machine matches that of action a_t or the job operation corresponds to a_t. The maximum number of actions to be checked is $\mathbb{m} + \mathbb{n}$. Based on Proposition 1 and the analysis of **CD2**, we now introduce Version II of the Policy Improvement Theorem.

Theorem 2 (Policy Improvement-Version II): By adopting the same assumptions as in Theorem 1, there exists a constant $\rho \in (0,1)$ such that if $\eta \in (0,\rho)$, there exists $\xi \in (0,1)$ such that $V^{\pi'}(s_t) > V^\pi(s_t)$ for all $\delta \in (\xi, 1)$, provided that the following conditions hold.

- $1 - \frac{\eta}{1-\eta} > \lambda + \kappa$.
- For any $a \in \mathcal{A}_m(s_t) \cup \mathcal{A}_{(j,i)}(s_t)$, after executing action a, the system clock of the resulting state remains the same as $\mathbb{T}_t$, which is the system clock of s_t.

Theorem 2 is particularly useful for algorithm design, as its conditions can be efficiently verified. While it ensures policy improvement under specific, checkable conditions, it requires collecting an entire trajectory before performing an update. This process can be computationally expensive,

Authorized licensed use limited to: Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on April 08,2026 at 12:30:36 UTC from IEEE Xplore. Restrictions apply.

especially for large-scale FJSP instances, which can lead to a lower convergence rate. Notice that, if the condition $r(s_t, a_t) + V^\pi(s_{t+1}) > V^\pi(s_t)$ holds, then there must exist a trajectory τ starting with (s_t, a_t) such that $G(\tau) > V^\pi(s_t)$. By applying Theorem 2, we immediately derive the following corollary:

Corollary 1: Given a policy π with the corresponding *action-priority vector* Φ, suppose that at state s_t, the selected action is a_t. If $r(s_t, a_t)+V^\pi(s_{t+1}) > V^\pi(s_t)$, then the same result as stated in Theorem 2 holds.

C. Interpretation

In this section, we provide insights into the Action-Priority Driven Policy Optimization algorithm, guided by the following key question: **What exactly does the algorithm do when solving the FJSP problem?**

Given an FJSP instance, we employ a linear policy representation, formulated as an *action-priority vector* Φ, whose length equals to the total number of actions. This vector assigns a priority to each action, while the state only determine the set of feasible actions. The state and Φ defines a probability distribution over these feasible actions. When solving FJSP, an associated *action-priority vector* Φ is constructed, and then we can define a mapping from an FJSP instance to Φ by setting $\phi_a = \frac{1}{(\mathcal{P})_{m,j,i}}$, where action $a = (m, j, i)$ and $(\mathcal{P})_{m,j,i}$ denotes the corresponding processing time. This mapping is based on the intuition that actions with longer processing times should be assigned lower priority values. Importantly, the mapping is bijective, meaning that a given *action-priority vector* Φ uniquely determines an FJSP instance, where the processing time of an action a is given by the reciprocal of its priority, i.e., $\frac{1}{\phi_a}$. During the policy update process, the *action-priority vector* Φ is modified to Φ'. From FJSP instance perspective, this update corresponds to an implicit adjustment of the processing times in the FJSP instance. **Thus, the algorithm iteratively refines the processing times of the FJSP instance, which is equivalent to solve a modified problem instance for which the untrained deterministic policy μ, as defined in (2), achieves improved performance**.

IV. Numerical Results

In this section, we evaluate the performance of the proposed algorithm using benchmark datasets from Brandimarte [12] and Hurink [4], both widely recognized for assessing solutions to the FJSP problem. The best-known upper bounds (UB) and theoretical lower bounds (LB) for these datasets, as reported by Behnke et al. [23], are used as reference values. The LB values are derived following the methodology introduced by Mastrolilli [24]. All experiments were performed on a system with the following specifications: Windows 10 operating system, Intel i7-9700F processor, 32 GB RAM, and an NVIDIA RTX 2070 Super GPU (8GB). Each problem instance from Brandimarte [12] was trained for 100 iterations, corresponding to the sampling of 100 trajectories, while instances from Hurink [4] underwent 300 training iterations. Table I summarizes the results, showing that APDPO achieves superior optimality (closer to OR-tools) and lower inference time compared with state-of-the-art RL methods. Alg. 1 provides the scheme and the pipeline of the algorithm. A detailed breakdown of the makespan for each instance is provided in Section IV of the supplementary document.

V. Conclusions

In this work, we introduce an Action-Priority Driven Policy Optimization method, in which the policy is represented by a linear *action-priority vector* and updated in a gradient-free manner. The proposed update rule guides the algorithm to increase or decrease the priority by checking whether or not the corresponding action makes any cumulative reward improvement. Furthermore, we establish a policy improvement theorem that theoretically guarantees policy improvement under specific and trackable conditions. The proposed algorithm efficiently addresses the Flexible Job Shop Scheduling Problem and surpasses state-of-the-art reinforcement learning methods, as demonstrated by numerical experiments on two benchmark datasets. Additionally, we provide an intuitive interpretation of the algorithm's mechanism. In future work, we will analyze the update rule for the case $G(\tau) < V^\pi(s_t)$ to enhance the data efficiency of the algorithm, and further examine its convergence behavior.

Alg. 1: Action-Priority Driven Policy Optimization

Input : Updating step size $\delta \in (0, 1)$, learning rate α, max episode N, max horizon T

Init : $\Phi = [\phi_1, \cdots, \phi_{|\mathcal{A}|}]^\top$ and parameters of the value network θ, replay buffer $\mathcal{D}$.

for $episode = 1$ to N **do**
  Start a new episode; let $\mathcal{D}, \mathcal{G}, \mathcal{P} = \{\}$
  Receive initial state s_0
  for $t = 0$ to $T - 1$ **do**
    Sample action $a_t \sim \pi(a|s_t) = \frac{\phi_{a_t}}{\sum_{a'} \phi_{a'} \mathbb{I}(s_t, a')}$
    Perform the action and get the next state s_{t+1}
    Store $(s_t, a_t, r_t, \pi(a_t|s_t))$ in $\mathcal{D}$
  end
  for $t = T - 1$ to 0 **do**
    Compute and store $G_t = r_t + G_{t-1}$ in $\mathcal{G}$
    Compute and store $\mathrm{P}_t = \pi(a_t|s_t)\mathrm{P}_{t-1}$ in $\mathcal{P}$
    Let $\eta = 1 - \pi(a_t, s_t)$, compute λ and κ based on Proposition 1
    if $G_t > V(s_t)$ **then**
      if $1 - \frac{\eta}{1-\eta} > \lambda + \kappa$ & ***CD2*** *satisfied* **then**
        $\phi'_a = \begin{cases} \phi_a + (1-\delta^-) \sum_{a' \neq a} \mathbb{I}(s_t, a') \phi_{a'}, & \text{if } a = a_t \\ \delta \phi_a, & \text{otherwise} \end{cases}$
  end
  Update estimated value-function by minimizing the loss $\mathcal{L}(\theta)$:
  $\mathcal{L}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \left(V_\theta(s) - (r + V_\theta(s'))\right)^2$
end

Output: The *action-priority vector* Φ

Authorized licensed use limited to: Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on April 08,2026 at 12:30:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I

PERFORMANCE ON PUBLIC BENCHMARKS

Benchmark		HGNN (greedy) [15] 10×5	HGNN (greedy) [15] 15×10	DANIEL (greedy) [16] 10×5	DANIEL (greedy) [16] 15×10	**Ours**	**Reduced**[1]	OR-Tools[2]	2SGA[14]	LB[23]	UB[23]
mk	$C_{\max}$	201.40	198.50	185.70	[184.40]	**178.10**	**6.30**	173.90	175.20	163.30	172.50
	Time (s)	1.26	1.25	1.29	1.30	0.79	0.46	2625.78	57.60	–	–
la (rdata)	$C_{\max}$	[1030.83]	1031.33	1031.63	1040.05	**961.53**	**69.30**	933.53	–	923.24	934.28
	Time (s)	1.40	1.40	1.37	1.36	0.95	0.41	2531.67	–	–	–
la (edata)	$C_{\max}$	1187.48	1182.08	1194.98	[1175.53]	**1059.05**	**116.48**	1026.80	–	1005.03	1029.48
	Time (s)	1.40	1.40	1.37	1.38	0.94	0.43	1637.05	–	–	–
la (vdata)	$C_{\max}$	955.90	954.33	[944.85]	948.73	**926.45**	**18.40**	920.40	812.20[3]	918.80	919.65
	Time (s)	1.37	1.37	1.37	1.37	1.05	0.32	1152.87	51.43	–	–

[1] The improvement of our methods versus other best DRL-based methods.
[2] Newly calculated versus [16] with 3600 seconds limit; actual CPU time (s) provided.
[3] The average makespan of 2SGA on la (vdata) benchmark is computed on la1-30 instances [14].

VI. PROOFS

A. Proof Sketch of Theorem 1

Before proving the theorem, we introduce key notations, lemmas, and a proposition essential to the proof, all derived based on the content of Theorem 1. Let $\mathcal{T}$ denote the set of all possible trajectories originating from state s_t. Define t_τ^+ as the first stage in τ where $\mathbb{T}'_{t^+} > \mathbb{T}'_{t^+-1}$, meaning $\mathbb{T}'_t = \mathbb{T}'_{t-1}$ for all $1 \le t \le t^+ - 1$. The set $\mathcal{T}$ is partitioned into five disjoint subsets: $\mathcal{T}_1$, $\mathcal{T}_2$, $\mathcal{T}_3$, $\mathcal{T}_4$, and $\mathcal{T}_5$. This decomposition is crucial because, after updating the policy via (3), the expected return increases for trajectories in $\mathcal{T}_1$, $\mathcal{T}_2 \cup \mathcal{T}_3$, and $\mathcal{T}_4$, while it decreases for $\mathcal{T}_5$. By appropriately choosing δ, we ensure that the net effect is an overall increase in the expected return of $\mathcal{T}$, thereby improving policy performance. Detailed descriptions of these subsets follow.

The set $\mathcal{T}_5$ consists solely of the trajectory τ', i.e., $\mathcal{T}_5 = \{\tau'\}$. The set $\mathcal{T}_4$ includes all trajectories where the first $t_{\tau'}^+ - 1$ actions are identical to those in $\{a_t, \ldots, a'_{t_{\tau'}^+ + t - 1}\}$, and the remaining actions follow the same sequence as τ', except for τ' itself. Formally, $\mathcal{T}_4 = \left\{\tau \middle| \mathcal{A}_{t_\tau^+}(\tau) = \mathcal{A}_{t_{\tau'}^+}(\tau'), \tau|s'_{t_{\tau'}^+} = \tau'|s'_{t_{\tau'}^+}, \tau \neq \tau', \tau \in \mathcal{T}\right\}$, where $\mathcal{A}_{t_\tau^+}(\tau)$ denotes the set of actions in τ before stage t_τ^+, and $\tau|s'_{t_{\tau'}^+}$ represents the truncated trajectory of τ after state $s'_{t_{\tau'}^+}$. Since $\mathcal{A}_{t_\tau^+}(\tau) = \mathcal{A}_{t_{\tau'}^+}(\tau')$ implies $t_\tau^+ = t_{\tau'}^+$ by the Permutation Irrelevant property (Proposition 2), we directly use $\tau|s'_{t_{\tau'}^+} = \tau'|s'_{t_{\tau'}^+}$. The set $\mathcal{T}_3$ includes all trajectories that begin with (s_t, a_t) except for τ', formally defined as $\mathcal{T}_3 = \{\tau \mid \tau = (s_t, a_t, \ldots), \tau \in \mathcal{T}\}$. The set $\mathcal{T}_2$ contains trajectories that share the same first $t_{\tilde{\tau}}^+$ actions with some $\tilde{\tau} \in \mathcal{T}_3$ but are distinct from both $\tilde{\tau}$ and all elements in $\mathcal{T}_3$: $\mathcal{T}_2 = \left\{\tau \middle| \exists \tilde{\tau} \in \mathcal{T}_3, \text{ s.t. } \mathcal{A}_{t_{\tilde{\tau}}^+}(\tau) = \mathcal{A}_{t_{\tilde{\tau}}^+}(\tilde{\tau}), \tau \neq \tilde{\tau}, \tau \notin \mathcal{T}_3, \tau \in \mathcal{T}\right\}$. Finally, the set $\mathcal{T}_1$ is given by $\mathcal{T}_1 = \mathcal{T} \backslash (\mathcal{T}_2 \cup \mathcal{T}_3 \cup \mathcal{T}_4 \cup \mathcal{T}_5)$. We define $\eta = \frac{\sum_{j \neq a_t} \mathbb{I}(s_t, j)\phi_j}{\sum_j \mathbb{I}(s_t, j)\phi_j}$, which represents the probability of selecting an action other than a_t in state s_t. The following propositions and lemmas will be introduced next.

Proposition 2 (Permutation Irrelevant): Given that some sequence $(s_0, a_0, \cdots, s_t, a_t, s_{t+1})$ is a state-action sequence with $\mathbb{T}_0 = \mathbb{T}_t < \mathbb{T}_{t+1}$, meaning that the system clock of states s_0 and s_t are the same and both are less than that of s_{t+1}. Then, consider any permutation of the action set $\{a_0, \ldots, a_t\}$, which forms a new ordered action sequence. If we execute these actions in the new given order, the resulting state will be exactly s_{t+1}.

Suppose $G(\tau') > V^\pi(s_t)$, for all actions $a \neq a_t$, we observe that $\pi'(a|s) \le \pi(a|s)$. Furthermore, once action a_t is executed, it is no longer feasible, meaning that $\pi(a|s) = \pi'(a|s)$ for all a, s where $a_t \notin \mathcal{A}(s)$. This observation is formally stated in Lemma 1.

Lemma 1: Given a policy π and its corresponding *action-priority vector* Φ, suppose $G(\tau') > V^\pi(s_t)$, then for the new policy π' and its corresponding Φ' constructed according to (3), we have following relations:

- $\pi(a|s) = \pi'(a|s), \forall a \neq a_t, a \in \mathcal{A}(s)$, if a_t is invalid for state s.
- $\pi'(a|s) < \pi(a|s), \forall a \neq a_t, a \in \mathcal{A}(s)$, if a_t is valid for state s. More specifically, if s is the state that satisfies $\mathcal{A}(s) \subseteq \mathcal{A}(s_t)$, then $\pi'(a|s) \le \delta\pi(a|s)$ with equality holds when $\mathcal{A}(s) = \mathcal{A}(s_t)$.

For the case $G(\tau') < V^\pi(s_t)$, the first result still holds, while the second result changes the inequality $\le$ to $\ge$.

Lemma 2 establishes that if Condition **CD2** holds, then for any trajectory τ in $\mathcal{T}$, at least two actions must be executed before the system clock advances for the first time.

Lemma 2: Suppose $\mathcal{A}_m(s_t)$ and $\mathcal{A}_{j,i}(s_t)$ satisfying **CD2** and $\tau \in \mathcal{T}$, then $t_\tau^+ \ge 2$.

Lemma 3 demonstrates the relationship between the probability of a trajectory τ from the set $\mathcal{T}_1$ after the update (i.e., $\mathrm{P}_\tau^{\pi'}$) and before (i.e., P_τ^π). Specifically, it shows $\mathrm{P}_\tau^{\pi'}$ is bounded above by a term involving δ multiplied by P_τ^π.

Lemma 3: Suppose $\tau \in \mathcal{T}_1$, then $a_t \notin \mathcal{A}_{t_\tau^+}(\tau)$. If π' is updated according to update rule (3), then for any $s \in \tau$

$$\mathrm{P}_\tau^{\pi'} < \frac{\delta^2(1-\delta\eta)\sum_j \mathbb{I}_j(s)\phi_j}{(1-\eta)(\delta\sum_j \mathbb{I}_j(s)\phi_j + (1-\delta)\sum_j \mathbb{I}(s_t, j)\phi_j)}\mathrm{P}_\tau^\pi$$

where $\mathrm{P}_\tau^{\pi'}$ represents the probability of τ given policy π'. π' is the new policy and π is the old one.

Authorized licensed use limited to: Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on April 08,2026 at 12:30:36 UTC from IEEE Xplore. Restrictions apply.

Lemma 4 asserts that if a trajectory τ from set $\mathcal{T}$ takes action a_t before the first system clock change at stage t, the probability of τ after the update, $\mathrm{P}_\tau^{\pi'}$, is bounded by a term involving t and δ, multiplied by the probability before the update, P_τ^{π}.

Lemma 4: Suppose $\tau \in \mathcal{T}$, $a_t \in \mathcal{A}_{t_\tau^+}(\tau)$ and a_t is executed at stage t, we know $0 \le t < t_\tau^+$. Then,

$$\mathrm{P}_\tau^{\pi'} < \frac{\delta^t(1-\delta\eta)}{1-\eta}\mathrm{P}_\tau^{\pi}$$

Lemma 5 provides an alternative characterization for the sets $\mathcal{T}_2$ and $\mathcal{T}_4$. $\mathcal{T}_2$ can be expressed as the union of multiple subsets, each derived from a trajectory in $\mathcal{T}_3$. Similarly, the set $\mathcal{T}_4$ can be represented as a set derived from τ'.

Lemma 5: Assume $\tilde{\tau} := (s_t, a_t, \tilde{s}_1, \tilde{a}_1, \ldots, \tilde{s}_{t_{\tilde{\tau}}^+}, ...) \in \mathcal{T}_3$, and the set $\{a_t, \tilde{a}_1, \ldots, a_{t_{\tilde{\tau}}^+-1}\}$ contains all the actions taken before the first change in the system clock within trajectory $\tilde{\tau}$. Next, define $\tilde{\tau}^{(t)}$ as the trajectory whose first $t_{\tilde{\tau}}^+$ actions match the set $\{a_t, \tilde{a}_1, \ldots, a_{t_{\tilde{\tau}}^+-1}\}$, and satisfy $\tilde{\tau}^{(t)}|\tilde{s}_{t_{\tilde{\tau}}^+} = \tilde{\tau}|\tilde{s}_{t_{\tilde{\tau}}^+}$. By the existence of the Permutation Irrelevant Property (Proposition 2), the equality $\tilde{\tau}^{(t)}|\tilde{s}_{t_{\tilde{\tau}}^+} = \tilde{\tau}|\tilde{s}_{t_{\tilde{\tau}}^+}$ is well-defined. The order of the first $t_{\tilde{\tau}}^+$ actions in $\tilde{\tau}^{(t)}$ is given by

$$a_{t'}^{(t)} = \begin{cases} \tilde{a}_1, & \text{if } t' = 0, \\ a_t, & \text{if } t' = t, \\ \tilde{a}_{t'+1}, & \text{if } 0 < t' < t, \\ \tilde{a}_{t'}, & \text{if } t' > t, \end{cases} \tag{5}$$

where $0 \le t \le t_{\tilde{\tau}}^+ - 1$. Then, an equivalent expression for the set $\mathcal{T}_2$ is given by

$$\mathcal{T}_2 = \bigcup_{\tilde{\tau} \in \mathcal{T}_3} \{\tilde{\tau}^{(0)}, \ldots, \tilde{\tau}^{(t_{\tilde{\tau}}^+-1)}\}.$$

Similarly, the set $\mathcal{T}_4$ can be expressed in the same form. In other words, $\mathcal{T}_4$ is a special case of this relation, since $\mathcal{T}_5$ contains only a single element, τ'. Specifically, we have

$$\mathcal{T}_4 = \{\tau'^{(0)}, \ldots, \tau'^{(t_{\tau'}^+-1)}\}.$$

where $\tau'^{(t)}$ for $0 \le t \le t_{\tau'}^+ - 1$ is defined similarly, satisfying $\tau'^{(t)}|s'_{t_{\tau'}^+} = \tau'|s'_{t_{\tau'}^+}$.

Then, we can continue our proof sketch for Theorem 1. The value function $V^\pi(s_t)$ can be divided into five parts, which correspond to the expected returns of trajectories in the sets $\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, \mathcal{T}_4$, and $\mathcal{T}_5$. By **CD1** and Lemma 3, the expected return of trajectories in $\mathcal{T}_1$ increases. By Lemma 5 and **CD1**, the expected return of trajectories in $\mathcal{T}_2 \cup \mathcal{T}_3$ also increases. By Lemma 4, the expected return of trajectories in $\mathcal{T}_4$ increases as well. However, the expected return of trajectories in $\mathcal{T}_5$ decreases. Since $G(\tau') > V^\pi(s_t)$, we can show that the total expected return across all five sets increases, which implies that $V^{\pi'}(s_t) > V^\pi(s_t)$. See Subsection III.I in the supplement material for details. ■

REFERENCES

[1] T. Borreguero-Sanchidrián, R. Pulido, Á. García-Sánchez, and M. Ortega-Mier, "Flexible job shop scheduling with operators in aeronautical manufacturing: A case study," *IEEE Access*, vol. 6, pp. 224–233, 2017.

[2] D. A. Rossit, F. Tohmé, and M. Frutos, "Industry 4.0: smart scheduling," *International Journal of Production Research*, vol. 57, no. 12, pp. 3802–3813, 2019.

[3] P. Brucker and R. Schlie, "Job-shop scheduling with multi-purpose machines," *Computing*, vol. 45, no. 4, pp. 369–375, 1991.

[4] J. Hurink, B. Jurisch, and M. Thole, "Tabu search for the job-shop scheduling problem with multi-purpose machines," *Operations-Research-Spektrum*, vol. 15, pp. 205–215, 1994.

[5] S. Dauzère-Pérès and J. Paulli, "An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search," *Annals of Operations research*, vol. 70, no. 0, pp. 281–306, 1997.

[6] M. R. Garey, D. S. Johnson, and R. Sethi, "The complexity of flowshop and jobshop scheduling," *Mathematics of Operations Research*, vol. 1, no. 2, pp. 117–129, 1976.

[7] S. Dauzère-Pérès, J. Ding, L. Shen, and K. Tamssaouet, "The flexible job shop scheduling problem: A review," *European Journal of Operational Research*, vol. 314, no. 2, pp. 409–432, 2024.

[8] J. Carlier and É. Pinson, "An algorithm for solving the job-shop problem," *Management Science*, vol. 35, no. 2, pp. 164–176, 1989.

[9] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," Gurobi, 2024.

[10] L. Perron and F. Didier, "CP-SAT," Google, 2024.

[11] J. Adams, E. Balas, and D. Zawack, "The shifting bottleneck procedure for job shop scheduling," *Management Science*, vol. 34, no. 3, pp. 391–401, 1988.

[12] P. Brandimarte, "Routing and scheduling in a flexible job shop by tabu search," *Annals of Operations research*, vol. 41, no. 3, pp. 157–183, 1993.

[13] G. Zhang, L. Gao, and Y. Shi, "An effective genetic algorithm for the flexible job-shop scheduling problem," *Expert Systems with Applications*, vol. 38, no. 4, pp. 3563–3573, 2011.

[14] D. Rooyani and F. M. Defersha, "An efficient two-stage genetic algorithm for flexible job-shop scheduling," *IFAC-PapersOnLine*, vol. 52, no. 13, pp. 2519–2524, 2019.

[15] W. Song, X. Chen, Q. Li, and Z. Cao, "Flexible job-shop scheduling via graph neural network and deep reinforcement learning," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1600–1610, 2023.

[16] R. Wang, G. Wang, J. Sun, F. Deng, and J. Chen, "Flexible job shop scheduling via dual attention network-based reinforcement learning," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–12, 2023.

[17] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, J. Pineau *et al.*, "An introduction to deep reinforcement learning," *Foundations and Trends® in Machine Learning*, vol. 11, no. 3-4, pp. 219–354, 2018.

[18] I. G. Smit, J. Zhou, R. Reijnen, Y. Wu, J. Chen, C. Zhang, Z. Bukhsh, Y. Zhang, and W. Nuijten, "Graph neural networks for job shop scheduling problems: A survey," *Computers & Operations Research*, vol. 176, p. 106914, 2025.

[19] Y. Feng, L. Zhang, Z. Yang, Y. Guo, and D. Yang, "Flexible job shop scheduling based on deep reinforcement learning," in *Asian Conference on Artificial Intelligence Technology*. IEEE, 2021, pp. 660–666.

[20] J. Yan, Z. Liu, T. Zhang, and Y. Zhang, "Autonomous decision-making method of transportation process for flexible job shop scheduling problem based on reinforcement learning," in *International Conference on Machine Learning and Intelligent Systems Engineering*, 2021, pp. 234–238.

[21] K. Lei, P. Guo, W. Zhao, Y. Wang, L. Qian, X. Meng, and L. Tang, "A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem," *Expert Systems with Applications*, vol. 205, p. 117796, 2022.

[22] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," *Advances in neural information processing systems*, vol. 12, 1999.

[23] D. Behnke and M. J. Geiger, "Test instances for the flexible job shop scheduling problem with work centers," *Arbeitspapier / Research Report*, 2012.

[24] M. Mastrolilli and L. M. Gambardella, "Effective neighbourhood functions for the flexible job shop problem," *Journal of Scheduling*, vol. 3, no. 1, pp. 3–20, 2000.

Authorized licensed use limited to: Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on April 08,2026 at 12:30:36 UTC from IEEE Xplore. Restrictions apply.